topic_4_layered_architecture

Theory, Edge Cases & Code Examples

- ✓ What is Layered Architecture?
- ✓ Simple Explanation: Building with Floors Analogy
- ✓ Classic 3-Tier Architecture
- ✓ Layer Responsibilities
- ✓ Presentation Layer (UI)

→ Swipe to learn →

topic_4_layered_architecture

Theory

📌 What is Layered Architecture?

Layered Architecture is a design pattern where you organize code into **horizontal layers**, where each layer has a specific responsibility and can only communicate with adjacent layers (usually the layer directly below).

Key Idea: Like a building with floors, each layer sits on top of another. Upper floors depend on lower floors, but lower floors don't depend on upper floors.

---

topic_4_layered_architecture

Theory

📌 Simple Explanation: Building with Floors Analogy

Imagine an office building:

Floor	Purpose	Can Access
3rd Floor: Offices	Where people work (User Interface)	Can go down to 2nd floor for supplies
2nd Floor: Supplies	Storage, resources (Business Logic)	Can go down to 1st floor for infrastructure
1st Floor: Infrastructure	Electricity, water, HVAC (Database)	Foundation - doesn't need upper floors

Key Rules:
- ✅ 3rd floor can use 2nd floor services
- ✅ 2nd floor can use 1st floor services
- ❌ 1st floor **CANNOT** use 3rd floor (circular dependency!)
- ❌ 3rd floor **SHOULD NOT** directly access 1st floor (skip layers)

---

topic_4_layered_architecture

Theory

📌 Classic 3-Tier Architecture

The most common form:

```
┌──────────────────────────────────────┐
│   Presentation Layer (UI)            │  ← Users interact here
│   - Web pages, Mobile UI, Desktop    │
│   - Controllers, Views               │
└──────────────────────────────────────┘
              ↓ (uses)
┌──────────────────────────────────────┐
│   Business Logic Layer (BLL)         │  ← Core application logic
│   - Business rules, validation       │
│   - Domain models, Services          │
└──────────────────────────────────────┘
              ↓ (uses)
┌──────────────────────────────────────┐
│   Data Access Layer (DAL)            │  ← Talks to database
│   - Database queries, ORM            │
│   - Data models, Repositories        │
└──────────────────────────────────────┘
              ↓ (uses)
┌──────────────────────────────────────┐
│   Database (PostgreSQL, MySQL, etc.) │
└──────────────────────────────────────┘
```

---

topic_4_layered_architecture

Theory

📌 Layer Responsibilities

topic_4_layered_architecture

Theory

📌 1. Presentation Layer (UI)

What it does:
- Displays information to users
- Handles user input (clicks, forms, gestures)
- Formats data for display

What it does NOT do:
- ❌ Business logic (e.g., calculating prices)
- ❌ Database access

Examples:
- Web: HTML/CSS/JavaScript, React components
- Desktop: Qt widgets, WPF windows
- Mobile: Android activities, iOS view controllers

Code Example:

```
// Presentation Layer: Web Controller
class UserController {
    UserService userService;  // ← Depends on Business Layer

public:
    void displayUserProfile(int userId) {
        // 1. Get data from business layer
        User user = userService.getUserById(userId);

        // 2. Format for display
        std::cout << "Name: " << user.name << "\n";
        std::cout << "Email: " << user.email << "\n";

        // ❌ NEVER do this in presentation layer:
        // db.query("SELECT * FROM users WHERE id = " + userId);
    }
};
```

---

topic_4_layered_architecture

Theory

📌 2. Business Logic Layer (BLL) (1/2)

What it does:
- Contains **business rules** (e.g., "users must be 18+ to register")
- Validates data (e.g., "email must be valid format")
- Coordinates workflows (e.g., "checkout process: check stock → charge card → send email")

What it does NOT do:
- ❌ UI-specific logic (e.g., button colors)
- ❌ Direct database queries

Examples:
- User registration validation
- Price calculations (discounts, taxes)
- Order processing workflows

Code Example:

```
// Business Logic Layer: Service
class UserService {
    UserRepository userRepo;  // ← Depends on Data Layer

public:
    bool registerUser(const std::string& name, const std::string& email, int age) {
        // Business rule: Must be 18 or older
        if (age < 18) {
            std::cerr << "Error: Must be 18 or older\n";
            return false;
        }

        // Business rule: Email must be valid
        if (!isValidEmail(email)) {
            std::cerr << "Error: Invalid email format\n";
            return false;
        }

        // Business rule: Email must be unique
        if (userRepo.emailExists(email)) {
            std::cerr << "Error: Email already registered\n";
            return false;
// ...continues on next slide
```

📌 2. Business Logic Layer (BLL) (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        }

        // All checks passed, save user
        User user = {name, email, age};
        userRepo.save(user);
        return true;
    }

private:
    bool isValidEmail(const std::string& email) {
        return email.find('@') != std::string::npos;  // Simplified
    }
};
```

topic_4_layered_architecture

Theory

📌 3. Data Access Layer (DAL) (1/2)

What it does:
- Handles **all database interactions** (queries, inserts, updates, deletes)
- Abstracts database details from business logic
- Often uses **Repository pattern** or **ORM (Object-Relational Mapping)**

What it does NOT do:
- ❌ Business logic (e.g., validation)
- ❌ UI formatting

Examples:
- Repository classes (UserRepository, OrderRepository)
- ORMs (Entity Framework, Hibernate)
- Database connection management

Code Example:

```
// Data Access Layer: Repository
class UserRepository {
    DatabaseConnection db;

public:
    User getUserById(int id) {
        std::string query = "SELECT * FROM users WHERE id = " + std::to_string(id);
        ResultSet result = db.query(query);

        User user;
        user.id = result.getInt("id");
        user.name = result.getString("name");
        user.email = result.getString("email");
        return user;
    }

    void save(const User& user) {
        std::string query = "INSERT INTO users (name, email) VALUES ('"
                          + user.name + "', '" + user.email + "')";
        db.execute(query);
    }

// ...continues on next slide
```

📌 3. Data Access Layer (DAL) (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    bool emailExists(const std::string& email) {
        std::string query = "SELECT COUNT(*) FROM users WHERE email = '" + email + "'";
        ResultSet result = db.query(query);
        return result.getInt(0) > 0;
    }
};
```

topic_4_layered_architecture

Theory

📌 Complete Example: User Registration Flow (1/2)

```
User fills form (Presentation Layer)
         ↓
UserController.register()
         ↓
UserService.registerUser()  (Business Layer)
  - Validate age >= 18
  - Validate email format
  - Check email not already used ← calls Data Layer
         ↓
UserRepository.save()  (Data Layer)
         ↓
Database.INSERT INTO users...
```

Full Code:

```
// === PRESENTATION LAYER ===
class WebController {
    UserService userService;

public:
    void handleRegistrationForm(const std::string& name,
                                const std::string& email,
                                int age) {
        std::cout << "User submitting registration form...\n";

        bool success = userService.registerUser(name, email, age);

        if (success) {
            std::cout << "Registration successful! Welcome, " << name << "!\n";
        } else {
            std::cout << "Registration failed. Please try again.\n";
        }
    }
};

// === BUSINESS LOGIC LAYER ===
class UserService {
    UserRepository userRepo;

// ...continues on next slide
```

📌 Complete Example: User Registration Flow (2/2)

[Code continues across slides]

```
// ...continued from previous slide
public:
    bool registerUser(const std::string& name,
                      const std::string& email,
                      int age) {
        // Business rules
        if (age < 18) return false;
        if (!isValidEmail(email)) return false;
        if (userRepo.emailExists(email)) return false;

        User user = {0, name, email, age};
        userRepo.save(user);
        return true;
    }
};

// === DATA ACCESS LAYER ===
class UserRepository {
    DatabaseConnection db;

public:
    void save(const User& user) {
        db.execute("INSERT INTO users ...");
    }

    bool emailExists(const std::string& email) {
        auto result = db.query("SELECT COUNT(*) FROM users WHERE email = '" + email + "'");
        return result.getInt(0) > 0;
    }
};
```

Flow:

1. User fills form → **Presentation** validates input (client-side)
2. Form submitted → **Business** validates rules (age, email uniqueness)
3. **Data** saves to database
4. Result flows back up: Data → Business → Presentation → User

---

topic_4_layered_architecture

Theory

📌 Benefits of Layered Architecture

Benefit	Explanation
Separation of Concerns	Each layer has ONE job (UI, logic, data)
Testability	Can test business logic without UI or database
Replaceability	Swap layers (e.g., change database from MySQL to PostgreSQL)
Maintainability	Changes in one layer don't affect others
Parallel Development	Teams can work on different layers simultaneously
Reusability	Business logic can be reused (Web + Mobile use same services)

---

topic_4_layered_architecture

Theory

📌 Layering Rules (Dependency Direction)

Golden Rule: **Upper layers depend on lower layers, NEVER the reverse.**

```
✅ CORRECT:
Presentation → Business → Data
(UI depends on services, services depend on data)

❌ WRONG:
Data → Business → Presentation
(Database depends on UI? No!)
```

Common Violations:

```
// ❌ BAD: Data layer depends on Presentation layer
class UserRepository {
    void save(const User& user) {
        db.execute("INSERT ...");

        // WRONG! Data layer should NOT know about UI
        std::cout << "User " << user.name << " saved! (UI message)";
    }
};

// ✅ GOOD: Data layer returns result, Presentation layer displays
class UserRepository {
    bool save(const User& user) {
        db.execute("INSERT ...");
        return true;  // Just return success/failure
    }
};

class WebController {
    void handleRegistration() {
        if (userService.register(user)) {
            std::cout << "User saved!";  // UI layer handles display
        }
    }
};
```

topic_4_layered_architecture

Theory

📌 N-Tier Architecture (More Layers) (1/2)

Sometimes you need MORE than 3 layers:

```
┌─────────────────────┐
│  Presentation Layer │  (Web UI, Mobile App)
└─────────────────────┘
          ↓
┌─────────────────────┐
│  API / Service Layer │  (REST API, GraphQL)
└─────────────────────┘
          ↓
┌─────────────────────┐
│  Business Logic Layer │  (Domain services)
└─────────────────────┘
          ↓
┌─────────────────────┐
│  Data Access Layer  │  (Repositories, ORM)
└─────────────────────┘
          ↓
┌─────────────────────┐
│  Database           │
└─────────────────────┘
```

topic_4_layered_architecture

Theory

📌 N-Tier Architecture (More Layers) (2/2)

Example: E-Commerce System

```
┌──────────────────────┐
│  Web UI              │  ← Users browse products
└──────────────────────┘
           ↓
┌──────────────────────┐
│  API Gateway         │  ← REST endpoints (/products, /checkout)
└──────────────────────┘
           ↓
┌──────────────────────┐
│  Business Services   │  ← Order processing, inventory management
└──────────────────────┘
           ↓
┌──────────────────────┐
│  Domain Models       │  ← Product, Order, Customer entities
└──────────────────────┘
           ↓
┌──────────────────────┐
│  Data Repositories   │  ← Database queries
└──────────────────────┘
           ↓
┌──────────────────────┐
│  Database (PostgreSQL) │
└──────────────────────┘
```

topic_4_layered_architecture

Theory

📌 When to Use Layered Architecture

Use When	Don't Use When
✅ Clear separation needed (UI, logic, data)	❌ Simple CRUD app (overkill)
✅ Multiple frontends (Web + Mobile)	❌ Rapid prototyping (too much structure)
✅ Team specialization (frontend, backend, DB teams)	❌ High-performance systems (layer overhead)
✅ Long-term maintainability important	❌ Throw-away code
✅ Business logic is complex	❌ Microservices (different pattern)

topic_4_layered_architecture

Theory

📌 Real-World Examples

Example 1: Banking Application

```
Presentation Layer:
  - Mobile app (iOS/Android)
  - Web portal (React)

Business Logic Layer:
  - Account validation
  - Transaction processing
  - Fraud detection

Data Access Layer:
  - AccountRepository
  - TransactionRepository

Database:
  - Oracle database
```

Example 2: E-Commerce

```
Presentation:
  - Product catalog pages
  - Shopping cart UI
  - Checkout form

Business:
  - Inventory management (check stock)
  - Price calculations (discounts, taxes)
  - Order workflows (payment → fulfillment → shipping)

Data:
  - ProductRepository
  - OrderRepository
  - CustomerRepository

Database:
  - MySQL database
```

---

topic_4_layered_architecture

Theory

📌 Layered vs Other Architectures

Architecture	Structure	Use Case
Layered	Horizontal layers (UI → Logic → Data)	Traditional web apps, desktop apps
Event-Driven	Event bus + subscribers	Real-time systems, GUIs
Microservices	Independent services	Large distributed systems
ECS	Entities + Components + Systems	Games, simulations

topic_4_layered_architecture

Theory

📌 Advanced: Dependency Inversion Principle

Problem: Business layer depends on concrete Data layer implementations.

```
// ❌ Tight coupling
class UserService {
    MySQLUserRepository userRepo;  // ← Depends on concrete MySQL implementation
};
```

Solution: Depend on abstractions (interfaces).

```
// ✅ Loose coupling
class IUserRepository {  // Interface (abstract)
public:
    virtual void save(const User& user) = 0;
    virtual User getById(int id) = 0;
};

class UserService {
    IUserRepository* userRepo;  // ← Depends on interface, not concrete class

public:
    UserService(IUserRepository* repo) : userRepo(repo) {}
};

// Implementations
class MySQLUserRepository : public IUserRepository { /* ... */ };
class PostgreSQLUserRepository : public IUserRepository { /* ... */ };
class InMemoryUserRepository : public IUserRepository { /* ... */ };  // For testing!
```

Benefit: Can swap implementations without changing business logic!

---

topic_4_layered_architecture

Edge Cases

📌 1. Layer Violation (Skipping Layers) (1/2)

Problem: Presentation layer directly accesses Data layer, skipping Business layer.

Naive Code:

```
// ❌ BAD: UI directly queries database
class WebController {
    UserRepository userRepo;

public:
    void displayUserProfile(int userId) {
        User user = userRepo.getUserById(userId);  // ← Skipped Business Layer!
        std::cout << "Name: " << user.name << "\n";
    }
};
```

Why It's Bad:
- Business rules bypassed (e.g., "only show active users")
- Validation skipped
- Harder to test (UI depends on database)
- Changes to business logic require changing UI code

Correct Code:

```
// ✅ GOOD: UI goes through Business Layer
class WebController {
    UserService userService;  // ← Use Business Layer

public:
    void displayUserProfile(int userId) {
        User user = userService.getUserById(userId);  // ← Business layer handles rules
        std::cout << "Name: " << user.name << "\n";
    }
};

class UserService {
    UserRepository userRepo;

public:
// ...continues on next slide
```

topic_4_layered_architecture

Edge Cases

📌 1. Layer Violation (Skipping Layers) (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    User getUserById(int userId) {
        User user = userRepo.getUserById(userId);

        // Business rule: Only show active users
        if (!user.isActive) {
            throw std::runtime_error("User is inactive");
        }

        return user;
    }
};
```

Lesson: **ALWAYS go through adjacent layers.** Never skip layers.

---

topic_4_layered_architecture

Edge Cases

📌 2. Circular Dependencies (1/3)

Problem: Lower layer depends on upper layer, creating a cycle.

Naive Code:

```
// ❌ BAD: Circular dependency
class UserService {  // Business Layer
    UserRepository userRepo;

public:
    void registerUser(const User& user) {
        userRepo.save(user);
    }
};

class UserRepository {  // Data Layer
    UserService userService;  // ← WRONG! Data depends on Business

public:
    void save(const User& user) {
        db.execute("INSERT ...");

        // ❌ WRONG: Data layer calling business layer!
        userService.sendWelcomeEmail(user);
    }
};
```

Why It Fails:
- **Circular dependency**: Business → Data → Business
- Hard to test (can't mock dependencies)
- Tight coupling (changes ripple everywhere)

topic_4_layered_architecture

Edge Cases

📌 2. Circular Dependencies (2/3)

Solution: Use Callbacks or Events

```
// ✅ GOOD: Use callback
class UserRepository {
    std::function<void(const User&)> onUserSaved;  // Callback

public:
    void setOnUserSaved(std::function<void(const User&)> callback) {
        onUserSaved = callback;
    }

    void save(const User& user) {
        db.execute("INSERT ...");

        if (onUserSaved) {
            onUserSaved(user);  // Notify higher layer
        }
    }
};

class UserService {
    UserRepository userRepo;

public:
    UserService() {
        // Register callback
        userRepo.setOnUserSaved([this](const User& user) {
            sendWelcomeEmail(user);
        });
    }

    void registerUser(const User& user) {
        userRepo.save(user);  // Data layer will call callback
    }
};
```

topic_4_layered_architecture

Edge Cases

📌 2. Circular Dependencies (3/3)

Alternative: Event System

```
class UserRepository {
    EventBus* eventBus;

public:
    void save(const User& user) {
        db.execute("INSERT ...");
        eventBus->publish(UserSavedEvent{user});  // Publish event
    }
};

class UserService {
    void init() {
        eventBus->subscribe("UserSaved", [this](const Event& e) {
            sendWelcomeEmail(e.user);
        });
    }
};
```

Lesson: **Lower layers should NOT depend on upper layers.** Use callbacks or events.

---

topic_4_layered_architecture

Edge Cases

📌 3. God Classes (Too Much Logic in One Layer) (1/2)

Problem: Business layer becomes a "God Class" with thousands of lines.

Naive Code:

```
// ❌ BAD: 5000-line "UserService" class
class UserService {
    // User registration logic
    void registerUser() { /* 200 lines */ }

    // User login logic
    void loginUser() { /* 150 lines */ }

    // Password reset logic
    void resetPassword() { /* 100 lines */ }

    // Email verification logic
    void verifyEmail() { /* 80 lines */ }

    // Profile update logic
    void updateProfile() { /* 120 lines */ }

    // ... 50 more methods!
};
```

Why It's Bad:
- **Hard to maintain** (too much in one file)
- **Hard to test** (need to test everything together)
- **Poor cohesion** (unrelated functionality mixed)

📌 3. God Classes (Too Much Logic in One Layer) (2/2)

Solution 1: Split by Feature

```
// ✅ GOOD: Separate services by feature
class UserRegistrationService {
    void registerUser() { /* ... */ }
    void verifyEmail() { /* ... */ }
};

class UserAuthenticationService {
    void loginUser() { /* ... */ }
    void resetPassword() { /* ... */ }
};

class UserProfileService {
    void updateProfile() { /* ... */ }
    void deleteAccount() { /* ... */ }
};
```

Solution 2: Use Domain-Driven Design (DDD)

```
// Domain: User Management
class UserDomain {
    UserRegistrationService registrationService;
    UserAuthenticationService authService;
    UserProfileService profileService;
};

// Domain: Order Management
class OrderDomain {
    OrderCreationService creationService;
    OrderFulfillmentService fulfillmentService;
};
```

Lesson: **Keep classes focused.** Split large services into smaller, cohesive ones.

topic_4_layered_architecture

Edge Cases

📌 4. Testing Challenges (Coupled Layers) (1/3)

Problem: Can't test Business layer without database running.

Naive Code:

```
class UserService {
    MySQLUserRepository userRepo;  // ← Concrete dependency

public:
    bool registerUser(const User& user) {
        // Validation logic
        if (!isValidEmail(user.email)) return false;

        // Save to database
        userRepo.save(user);  // ← Requires MySQL running!
        return true;
    }
};

// Test
void testRegisterUser() {
    UserService service;
    User user = {"test@example.com", 25};

    bool result = service.registerUser(user);  // ❌ Fails if no database!
}
```

Why It's Bad:
- Tests require full database setup (slow!)
- Can't test business logic in isolation
- Tests break if database changes

📌 4. Testing Challenges (Coupled Layers) (2/3)

Solution: Dependency Injection + Mock

```
// ✅ GOOD: Abstract repository
class IUserRepository {
public:
    virtual void save(const User& user) = 0;
    virtual ~IUserRepository() = default;
};

class UserService {
    IUserRepository* userRepo;  // ← Abstract dependency

public:
    UserService(IUserRepository* repo) : userRepo(repo) {}

    bool registerUser(const User& user) {
        if (!isValidEmail(user.email)) return false;
        userRepo->save(user);
        return true;
    }
};

// Real implementation
class MySQLUserRepository : public IUserRepository {
    void save(const User& user) override {
        db.execute("INSERT INTO users ...");
    }
};

// Mock for testing
class MockUserRepository : public IUserRepository {
public:
    bool saveCalled = false;
    User savedUser;

    void save(const User& user) override {
        saveCalled = true;
        savedUser = user;
    }
};

// ...continues on next slide
```

📌 4. Testing Challenges (Coupled Layers) (3/3)

[Code continues across slides]

```
// ...continued from previous slide
// Test
void testRegisterUser() {
    MockUserRepository mockRepo;
    UserService service(&mockRepo);  // Inject mock

    User user = {"test@example.com", 25};
    bool result = service.registerUser(user);

    assert(result == true);
    assert(mockRepo.saveCalled == true);  // ✅ No database needed!
}
```

Lesson: **Use dependency injection.** Depend on abstractions, not concrete implementations.

---

topic_4_layered_architecture

Edge Cases

📌 5. Performance Overhead (Too Many Layers) (1/2)

Problem: Every request goes through 5+ layers, causing latency.

Naive Code:

```
// Request flow (7 layers!):
UI → Controller → API Gateway → Service → Domain → Repository → Database

// Each layer adds overhead:
void displayUser(int id) {
    controller.getUser(id);           // 1ms
        → apiGateway.getUser(id);     // 1ms
            → userService.getUser(id);    // 1ms
                → userDomain.getUser(id); // 1ms
                    → repository.getUser(id);  // 1ms
                        → database.query();    // 10ms
}
// Total: 15ms (5ms just from layers!)
```

Why It's Bad:
- **Latency overhead**: Each layer adds function call overhead
- **Over-engineering**: Too many abstraction layers for simple operations

Solution 1: Flatten for Simple Queries

```
// ✅ For simple CRUD, skip intermediate layers
void displayUser(int id) {
    User user = repository.getUserById(id);  // Direct access for simple query
    render(user);
}

// ✅ For complex operations, use layers
void registerUser(const User& user) {
    // Complex validation, business rules → use service layer
    userService.registerUser(user);
}
```

📌 5. Performance Overhead (Too Many Layers) (2/2)

Solution 2: Caching

```
class UserService {
    UserRepository userRepo;
    Cache cache;

public:
    User getUserById(int id) {
        // Check cache first
        if (cache.has(id)) {
            return cache.get(id);  // ✅ Fast path (no database)
        }

        // Cache miss → query database
        User user = userRepo.getUserById(id);
        cache.put(id, user);
        return user;
    }
};
```

Lesson: **Don't over-layer.** Use layers for complex logic, simplify for simple operations.

topic_4_layered_architecture

Edge Cases

📌 6. Cross-Cutting Concerns (Logging, Security) (1/3)

Problem: Every layer needs logging, authentication, error handling → code duplication.

Naive Code:

```
class UserController {
    void displayUser(int id) {
        log("Request: displayUser(" + std::to_string(id) + ")");  // ← Duplicate
        try {
            // ...
        } catch (...) {
            log("Error: displayUser failed");  // ← Duplicate
        }
    }
};

class UserService {
    void registerUser(const User& user) {
        log("Request: registerUser(" + user.name + ")");  // ← Duplicate
        try {
            // ...
        } catch (...) {
            log("Error: registerUser failed");  // ← Duplicate
        }
    }
};
```

Why It's Bad:
- **Code duplication**: Logging logic repeated everywhere
- **Hard to maintain**: Change logging format → update 100 places

📌 6. Cross-Cutting Concerns (Logging, Security) (2/3)

Solution 1: Aspect-Oriented Programming (Decorators)

```
// ✅ GOOD: Logging decorator
template<typename ServiceT>
class LoggingDecorator : public ServiceT {
    ServiceT* innerService;

public:
    LoggingDecorator(ServiceT* service) : innerService(service) {}

    auto registerUser(const User& user) {
        log("Request: registerUser(" + user.name + ")");
        try {
            auto result = innerService->registerUser(user);
            log("Success: registerUser");
            return result;
        } catch (...) {
            log("Error: registerUser failed");
            throw;
        }
    }
};

// Usage
UserService* baseService = new UserService();
UserService* loggedService = new LoggingDecorator(baseService);
```

📌 6. Cross-Cutting Concerns (Logging, Security) (3/3)

Solution 2: Middleware (For Web Apps)

```
class LoggingMiddleware {
public:
    void handle(Request request, Response response, NextHandler next) {
        log("Request: " + request.url);
        auto start = now();

        next(request, response);  // Call next handler

        auto duration = now() - start;
        log("Response: " + std::to_string(duration) + "ms");
    }
};

// Apply to all routes
app.use(LoggingMiddleware());
```

Lesson: **Use cross-cutting patterns** (decorators, middleware, AOP) for common concerns.

topic_4_layered_architecture

Code Examples

📌 1. Simple 3-Tier CRUD Application (1/7)

Goal: Implement basic User management (Create, Read, Update, Delete) with proper layering.

Output:

```
#include <iostream>
#include <vector>
#include <string>
#include <map>
#include <memory>

// ═══════════════════════════════════════════════════════════════
//  DATA MODELS
// ═══════════════════════════════════════════════════════════════

struct User {
    int id;
    std::string name;
    std::string email;
    int age;
};

// ═══════════════════════════════════════════════════════════════
//  DATA ACCESS LAYER (Repository Pattern)
// ═══════════════════════════════════════════════════════════════

class IUserRepository {
public:
    virtual void save(const User& user) = 0;
    virtual User getById(int id) = 0;
    virtual std::vector<User> getAll() = 0;
    virtual void update(const User& user) = 0;
    virtual void remove(int id) = 0;
    virtual bool emailExists(const std::string& email) = 0;
    virtual ~IUserRepository() = default;
};

class InMemoryUserRepository : public IUserRepository {
    std::map<int, User> users;
    int nextId = 1;

// ...continues on next slide
```

topic_4_layered_architecture

Code Examples

📌 1. Simple 3-Tier CRUD Application (2/7)

```
// ...continued from previous slide
public:
    void save(const User& user) override {
        User newUser = user;
        newUser.id = nextId++;
        users[newUser.id] = newUser;
        std::cout << "  [DATA] Saved user ID " << newUser.id << " to database\n";
    }

    User getById(int id) override {
        std::cout << "  [DATA] Querying user ID " << id << "\n";
        return users.at(id);
    }

    std::vector<User> getAll() override {
        std::cout << "  [DATA] Querying all users\n";
        std::vector<User> result;
        for (auto& [id, user] : users) {
            result.push_back(user);
        }
        return result;
    }

    void update(const User& user) override {
        std::cout << "  [DATA] Updating user ID " << user.id << "\n";
        users[user.id] = user;
    }

    void remove(int id) override {
        std::cout << "  [DATA] Deleting user ID " << id << "\n";
        users.erase(id);
    }

    bool emailExists(const std::string& email) override {
        for (auto& [id, user] : users) {
            if (user.email == email) return true;
        }
        return false;
    }
};

// ...continues on next slide
```

topic_4_layered_architecture

Code Examples

📌 1. Simple 3-Tier CRUD Application (3/7)

```
// ...continued from previous slide
// ═══════════════════════════════════════════════════════════════
//  BUSINESS LOGIC LAYER (Service)
// ═══════════════════════════════════════════════════════════════

class UserService {
    IUserRepository* userRepo;

public:
    UserService(IUserRepository* repo) : userRepo(repo) {}

    bool registerUser(const std::string& name, const std::string& email, int age) {
        std::cout << "\n[BUSINESS] Registering user: " << name << "\n";

        // Business rule: Age must be 18+
        if (age < 18) {
            std::cerr << "  ❌ Error: Must be 18 or older\n";
            return false;
        }

        // Business rule: Email must contain '@'
        if (email.find('@') == std::string::npos) {
            std::cerr << "  ❌ Error: Invalid email format\n";
            return false;
        }

        // Business rule: Email must be unique
        if (userRepo->emailExists(email)) {
            std::cerr << "  ❌ Error: Email already registered\n";
            return false;
        }

        std::cout << "  ✅ All validations passed\n";
        User user = {0, name, email, age};
        userRepo->save(user);
        return true;
    }

    User getUserById(int id) {
        std::cout << "\n[BUSINESS] Getting user ID " << id << "\n";
// ...continues on next slide
```

topic_4_layered_architecture

Code Examples

📌 1. Simple 3-Tier CRUD Application (4/7)

```
// ...continued from previous slide
        return userRepo->getById(id);
    }

    std::vector<User> getAllUsers() {
        std::cout << "\n[BUSINESS] Getting all users\n";
        return userRepo->getAll();
    }

    bool updateUserEmail(int id, const std::string& newEmail) {
        std::cout << "\n[BUSINESS] Updating email for user ID " << id << "\n";

        // Business rule: Email format validation
        if (newEmail.find('@') == std::string::npos) {
            std::cerr << "  ❌ Error: Invalid email format\n";
            return false;
        }

        User user = userRepo->getById(id);
        user.email = newEmail;
        userRepo->update(user);
        std::cout << "  ✅ Email updated\n";
        return true;
    }

    void deleteUser(int id) {
        std::cout << "\n[BUSINESS] Deleting user ID " << id << "\n";
        userRepo->remove(id);
    }
};

// ═══════════════════════════════════════════════════════════
//  PRESENTATION LAYER (Controller)
// ═══════════════════════════════════════════════════════════

class UserController {
    UserService* userService;

public:
    UserController(UserService* service) : userService(service) {}

// ...continues on next slide
```

topic_4_layered_architecture

Code Examples

📌 1. Simple 3-Tier CRUD Application (5/7)

```
// ...continued from previous slide
    void registerUser(const std::string& name, const std::string& email, int age) {
        std::cout << "\n═══════════════════════════════════════════════════\n";
        std::cout << "[UI] User submitting registration form\n";
        std::cout << "  Name: " << name << "\n";
        std::cout << "  Email: " << email << "\n";
        std::cout << "  Age: " << age << "\n";

        bool success = userService->registerUser(name, email, age);

        if (success) {
            std::cout << "\n[UI] ✅ Registration successful! Welcome, " << name << "!\n";
        } else {
            std::cout << "\n[UI] ❌ Registration failed. Please try again.\n";
        }
    }

    void displayUser(int userId) {
        std::cout << "\n═══════════════════════════════════════════════════\n";
        std::cout << "[UI] Displaying user profile\n";

        User user = userService->getUserById(userId);

        std::cout << "\n╔═══════════════════════════════════╗\n";
        std::cout << "\n╔═══════════════════════════════════╗\n";
        std::cout << "\n╚═══════════════════════════════════╝\n";
        std::cout << "  ID:    " << user.id << "\n";
        std::cout << "  Name:  " << user.name << "\n";
        std::cout << "  Email: " << user.email << "\n";
        std::cout << "  Age:   " << user.age << "\n";
    }

    void displayAllUsers() {
        std::cout << "\n═══════════════════════════════════════════════════\n";
        std::cout << "[UI] Displaying all users\n";

        std::vector<User> users = userService->getAllUsers();

        std::cout << "\n╔═══════════════════════════════════╗\n";
        std::cout << "\n╔═══════════════════════════════════╗\n";
        std::cout << "\n╚═══════════════════════════════════╝\n";
// ...continues on next slide
```

topic_4_layered_architecture

Code Examples

📌 1. Simple 3-Tier CRUD Application (6/7)

```
// ...continued from previous slide

        for (const User& user : users) {
            std::cout << "  " << user.id << ". " << user.name
                      << " (" << user.email << ")\n";
        }
    }
};

// ═══════════════════════════════════════════════════════════════
//  MAIN
// ═══════════════════════════════════════════════════════════════

int main() {
    std::cout << "╔═══════════════════════════════════════════════════╗\n";
    std::cout << "║  3-Tier Architecture: User Management System     ║\n";
    std::cout << "╚═══════════════════════════════════════════════════╝\n";

    // Setup layers (Dependency Injection)
    InMemoryUserRepository repository;
    UserService service(&repository);
    UserController controller(&service);

    // Test 1: Register valid user
    controller.registerUser("Alice", "alice@example.com", 25);

    // Test 2: Register underage user (should fail)
    controller.registerUser("Bob", "bob@example.com", 16);

    // Test 3: Register user with invalid email (should fail)
    controller.registerUser("Charlie", "charlie-invalid", 30);

    // Test 4: Register duplicate email (should fail)
    controller.registerUser("Dave", "alice@example.com", 28);

    // Test 5: Register another valid user
    controller.registerUser("Eve", "eve@example.com", 32);

    // Test 6: Display single user
    controller.displayUser(1);

// ...continues on next slide
```

topic_4_layered_architecture

Code Examples

📌 1. Simple 3-Tier CRUD Application (7/7)

[Code continues across slides]

```
// ...continued from previous slide
    // Test 7: Display all users
    controller.displayAllUsers();

    std::cout << "\n╔═══════════════════════════════════════════════════╗\n";
    std::cout << "│  Notice: Each layer has distinct responsibility  │\n";
    std::cout << "│  - UI: Display and format                         │\n";
    std::cout << "│  - Business: Validation and rules                 │\n";
    std::cout << "│  - Data: Database operations                      │\n";
    std::cout << "└───────────────────────────────────────────────────┘\n";

    return 0;
}
```

```
['╔═══════════════════════════════════════════════════╗\n│  3-Tier Architecture: User Management System     │\n└──────────
```

topic_4_layered_architecture

Code Examples

📌 2. E-Commerce Order System (4-Tier) (1/5)

Goal: Add an API layer between UI and Business for REST endpoints.

Flow:

```
// ═══════════════════════════════════════════════════════════════
//  4-TIER ARCHITECTURE
// ═══════════════════════════════════════════════════════════════
//
//  Presentation Layer (Web UI)
//       ↓
//  API Layer (REST endpoints)
//       ↓
//  Business Logic Layer (Order processing)
//       ↓
//  Data Access Layer (Database)
//
// ═══════════════════════════════════════════════════════════════

struct Product {
    int id;
    std::string name;
    double price;
    int stock;
};

struct Order {
    int id;
    int userId;
    std::vector<int> productIds;
    double totalPrice;
    std::string status;  // "pending", "completed", "cancelled"
};

// === DATA LAYER ===
class ProductRepository {
    std::map<int, Product> products;

public:
    ProductRepository() {
        products[1] = {1, "Laptop", 999.99, 10};
// ...continues on next slide
```

topic_4_layered_architecture

Code Examples

📌 2. E-Commerce Order System (4-Tier) (2/5)

```
// ...continued from previous slide
        products[2] = {2, "Mouse", 29.99, 50};
        products[3] = {3, "Keyboard", 79.99, 30};
    }

    Product getById(int id) {
        return products.at(id);
    }

    void updateStock(int id, int newStock) {
        products[id].stock = newStock;
    }
};

class OrderRepository {
    std::map<int, Order> orders;
    int nextId = 1;

public:
    void save(const Order& order) {
        Order newOrder = order;
        newOrder.id = nextId++;
        orders[newOrder.id] = newOrder;
        std::cout << "  [DATA] Saved order ID " << newOrder.id << "\n";
    }

    Order getById(int id) {
        return orders.at(id);
    }
};

// === BUSINESS LAYER ===
class OrderService {
    ProductRepository* productRepo;
    OrderRepository* orderRepo;

public:
    OrderService(ProductRepository* pRepo, OrderRepository* oRepo)
        : productRepo(pRepo), orderRepo(oRepo) {}

    bool createOrder(int userId, const std::vector<int>& productIds) {
// ...continues on next slide
```

📌 2. E-Commerce Order System (4-Tier) (3/5)

```
// ...continued from previous slide
        std::cout << "\n[BUSINESS] Processing order for user " << userId << "\n";

        double totalPrice = 0;

        // Business rule: Check stock and calculate price
        for (int productId : productIds) {
            Product product = productRepo->getById(productId);

            if (product.stock == 0) {
                std::cerr << "  ❌ " << product.name << " is out of stock!\n";
                return false;
            }

            totalPrice += product.price;
            std::cout << "  ✅ " << product.name << " ($" << product.price << ")\n";
        }

        // Business rule: Minimum order $20
        if (totalPrice < 20) {
            std::cerr << "  ❌ Minimum order is $20\n";
            return false;
        }

        // Create order
        Order order = {0, userId, productIds, totalPrice, "pending"};
        orderRepo->save(order);

        // Update stock
        for (int productId : productIds) {
            Product product = productRepo->getById(productId);
            productRepo->updateStock(productId, product.stock - 1);
        }

        std::cout << "  ✅ Order total: $" << totalPrice << "\n";
        return true;
    }

    Order getOrder(int orderId) {
        return orderRepo->getById(orderId);
// ...continues on next slide
```

📌 2. E-Commerce Order System (4-Tier) (4/5)

```
// ...continued from previous slide
    }
};

// === API LAYER ===
class OrderAPI {
    OrderService* orderService;

public:
    OrderAPI(OrderService* service) : orderService(service) {}

    // REST endpoint: POST /orders
    std::string createOrder(const std::string& requestBody) {
        std::cout << "\n[API] POST /orders\n";
        std::cout << "  Request body: " << requestBody << "\n";

        // Parse JSON (simplified)
        int userId = 1;
        std::vector<int> productIds = {1, 2};  // From request

        bool success = orderService->createOrder(userId, productIds);

        if (success) {
            return "{\"status\": \"success\", \"message\": \"Order created\"}";
        } else {
            return "{\"status\": \"error\", \"message\": \"Order failed\"}";
        }
    }

    // REST endpoint: GET /orders/:id
    std::string getOrder(int orderId) {
        std::cout << "\n[API] GET /orders/" << orderId << "\n";

        Order order = orderService->getOrder(orderId);

        return "{\"id\": " + std::to_string(order.id) +
               ", \"total\": " + std::to_string(order.totalPrice) +
               ", \"status\": \"" + order.status + "\"}";
    }
};

// ...continues on next slide
```

topic_4_layered_architecture

Code Examples

📌 2. E-Commerce Order System (4-Tier) (5/5)

[Code continues across slides]

```
// ...continued from previous slide
// === PRESENTATION LAYER ===
class WebUI {
    OrderAPI* api;

public:
    WebUI(OrderAPI* api) : this->api(api) {}

    void displayCheckoutPage() {
        std::cout << "\n═══════════════════════════════════════════════\n";
        std::cout << "[UI] Checkout Page\n";
        std::cout << "  User clicks 'Place Order' button...\n";

        std::string requestBody = "{\"userId\": 1, \"products\": [1, 2]}";
        std::string response = api->createOrder(requestBody);

        std::cout << "\n[UI] Server response: " << response << "\n";
    }
};

int main() {
    // Setup
    ProductRepository productRepo;
    OrderRepository orderRepo;
    OrderService orderService(&productRepo, &orderRepo);
    OrderAPI api(&orderService);
    WebUI ui(&api);

    // User workflow
    ui.displayCheckoutPage();

    return 0;
}
```

```
['User clicks button → WebUI.displayCheckoutPage()\n                    → OrderAPI.createOrder() (REST endpoint)\n
```

What You Learned

Key Takeaways

- ✓ What is Layered Architecture?
- ✓ Simple Explanation: Building with Floors Analogy
- ✓ Classic 3-Tier Architecture
- ✓ Layer Responsibilities

→ Swipe to learn →